



Web Application Security **Vulnerability Assessment &** **Secure Coding Implementation**

Prepared By: **Zahid Ali**

Submitted To: **Dr. Syed Muhammad Awais**

Table of Contents

1. Introduction	3
2. Environment Setup & System Architecture	4
3. Implementation Tasks Overview	5
Task 1: Environment Setup (Flask + Python + SQL Server)	5
Task 2: Database Creation (SQL Server)	6
Task 3: Basic Flask Application Development	6
Task 4: Login Form Implementation	7
Task 5: SQL Injection Vulnerability Demonstration	7
Task 6: Dashboard After SQL Injection	8
Task 7: Authentication Bypass Vulnerability	8
Task 8: Privilege Escalation Vulnerability	9
Task 9: SQL Injection Attack Testing	9
Task 10: Authentication Bypass Testing	10
Task 11: Privilege Escalation Testing	10
Task 12: HTTP Request Modification (Burp Suite)	11
Task 13: Vulnerability Assessment Report	11
Task 14: Secure Coding Fixes	12
Task 15: Application Re-Testing (Post-Fix)	13
Task 16: Final Vulnerability Dashboard & Reporting	13
4. Vulnerability Analysis Table	14
5. Attack Demonstration Details	15
6. Secure Coding Implementation	16
7. Final Secure Testing Results	17
8. Conclusion	18
9. References	19

1. Introduction

This laboratory report presents a comprehensive assessment of web application security vulnerabilities along with the implementation of secure coding practices in a controlled environment. The primary objective of this lab was to simulate real-world security flaws within a Flask-based web application and systematically identify, exploit, and mitigate critical vulnerabilities.

The assessment focuses on major web security threats, including SQL Injection, Authentication Bypass, Privilege Escalation, and HTTP Request Manipulation. Through practical penetration testing techniques, these vulnerabilities were intentionally introduced, analyzed, and exploited to demonstrate their potential impact on application integrity, confidentiality, and access control mechanisms.

Furthermore, this lab emphasizes the importance of secure software development by applying industry-standard mitigation techniques such as parameterized queries, session-based authentication, and strict server-side validation. The transition from a vulnerable system to a secure implementation highlights the effectiveness of defensive programming practices.

1.1 Purpose of the Lab

The primary objective of this laboratory exercise was to gain hands-on experience in identifying, exploiting, and remediating common web application vulnerabilities in a controlled environment. Using a Flask-based web application connected to a Microsoft SQL Server database, students simulated real-world attack scenarios and then applied industry-standard secure coding techniques to mitigate those vulnerabilities.

1.2 Importance of Web Application Security

Web applications are among the most frequently targeted assets in modern cyberattacks. According to the OWASP Top 10, injection attacks (including SQL Injection) and broken authentication consistently rank as the most critical vulnerabilities affecting web systems. A single exploited vulnerability can lead to unauthorized access, data breaches, privilege escalation, and complete system compromise.

1.3 Vulnerabilities Studied

The following categories of vulnerabilities were studied and demonstrated in this lab:

- SQL Injection (SQLi): Exploiting unsanitized input fields to manipulate database queries and bypass authentication.
- Authentication Bypass: Circumventing login mechanisms through logic flaws and missing server-side validation.
- Privilege Escalation: Gaining elevated system privileges by manipulating URL parameters or session data.
- HTTP Request Modification: Intercepting and modifying HTTP requests using tools such as Burp Suite to bypass controls.

1.4 Lab Environment

The lab utilized a dual-environment setup consisting of a Kali Linux virtual machine running the Flask application and a Windows host machine running Microsoft SQL Server. This architecture closely mimics real-world enterprise web application deployments.

2. Environment Setup & System Architecture

2.1 System Architecture Overview

The lab infrastructure consists of two primary components communicating over a shared network:

Component	Details	Role in Lab
Kali Linux VM	Python 3, Flask Framework, Virtual Environment	Hosts web application, simulates attacker machine
Windows Host	Microsoft SQL Server, SSMS	Hosts backend database, stores user records
Network	Local IP / Host-Only Adapter	Connects Flask app to SQL Server database
Browser	Firefox / Chromium on Kali	Used to access and test the web application
Burp Suite	Kali Linux proxy tool	HTTP request interception and modification

2.2 Architecture Diagram Description

The following diagram illustrates the logical architecture of the lab environment:

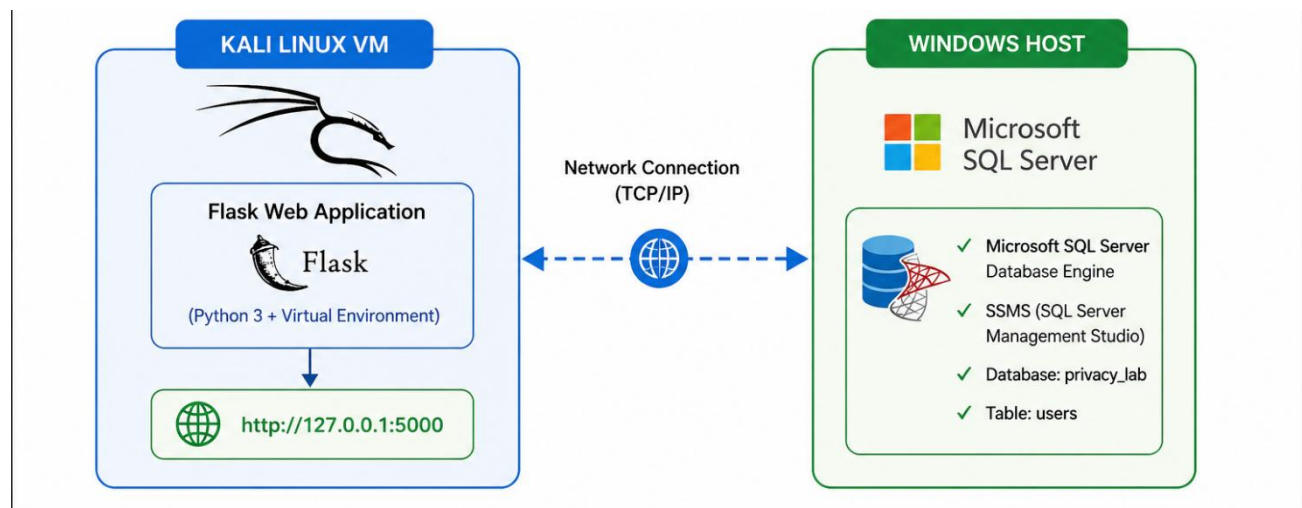


Figure 2.1: System Architecture – Kali Linux Flask App communicating with Windows SQL Server

3. Implementation Tasks

This section documents all 16 tasks performed during the lab, including objectives, implementation steps, observations, and screenshot references.

Task 1: Environment Setup (Flask + Python + SQL Server)

Objective

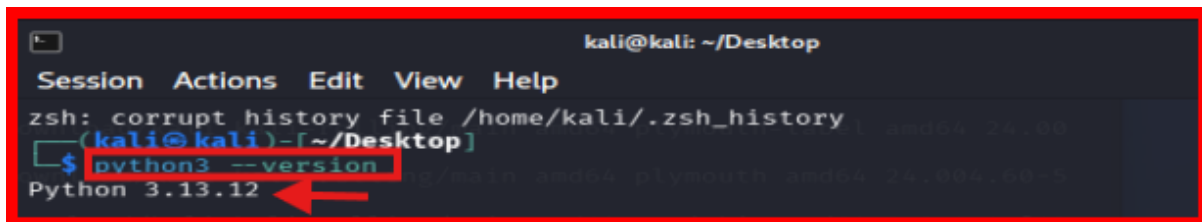
Configure the complete development environment including Python virtual environment, Flask framework installation, and initial project directory structure on Kali Linux.

Steps Performed

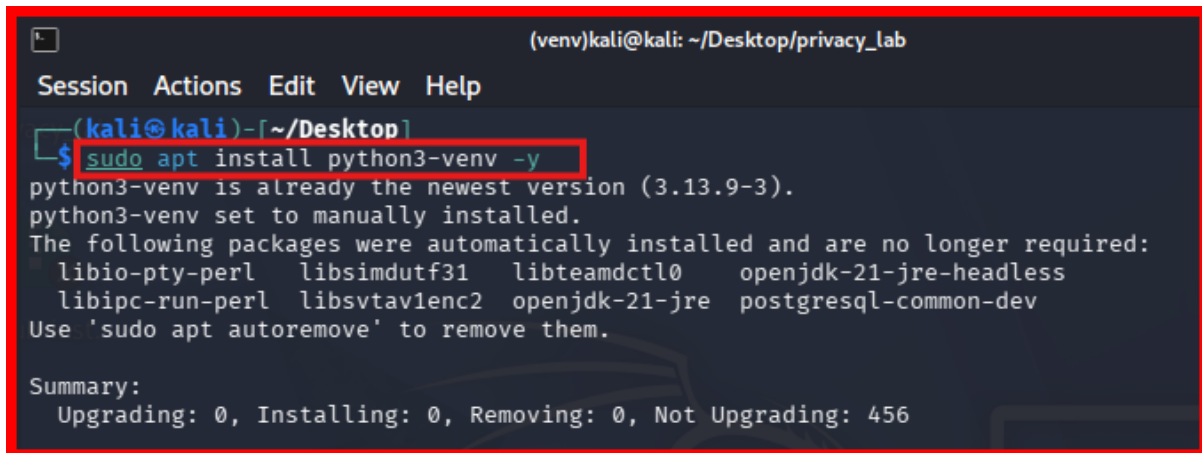
1. Verified Python 3 installation using `python3 --version` in the Kali Linux terminal.
2. Created a project directory: `mkdir privacy_lab && cd privacy_lab`
3. Created a Python virtual environment: `python3 -m venv venv`
4. Activated the virtual environment: `source venv/bin/activate`
5. Installed Flask using pip: `pip install flask`
6. Created the initial `app.py` file with a basic Flask route.
7. Launched the Flask development server: `python app.py`
8. Accessed the application in the browser at `http://127.0.0.1:5000`

Observation

The Flask development server started successfully and was accessible via the browser. The virtual environment isolated project dependencies from the system Python installation.



A terminal window titled 'kali@kali: ~/Desktop' with a menu bar 'Session Actions Edit View Help'. The prompt is '(kali@kali)-[~/Desktop]'. The command `$ python3 --version` is entered and highlighted with a red box. The output is 'Python 3.13.12', with a red arrow pointing to it.



A terminal window titled '(venv)kali@kali: ~/Desktop/privacy_lab' with a menu bar 'Session Actions Edit View Help'. The prompt is '(kali@kali)-[~/Desktop]'. The command `$ sudo apt install python3-venv -y` is entered and highlighted with a red box. The output shows that python3-venv is already installed (3.13.9-3) and lists several packages that were automatically installed and are no longer required, such as libio-pty-perl, libsimdutf31, libteamctl0, openjdk-21-jre-headless, libipc-run-perl, libsvtav1enc2, openjdk-21-jre, and postgresql-common-dev. A summary at the bottom shows 'Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 456'.

```
(kali@kali)-[~/Desktop]
$ mkdir ~/Desktop/privacy_lab
cd ~/Desktop/privacy_lab

(kali@kali)-[~/Desktop/privacy_lab]
$ python3 -m venv venv

(kali@kali)-[~/Desktop/privacy_lab]
$ source venv/bin/activate

(venv)-(kali@kali)-[~/Desktop/privacy_lab]
$
```

```
(venv)kali@kali: ~/Desktop/privacy_lab
Session Actions Edit View Help
(venv)-(kali@kali)-[~/Desktop/privacy_lab]
$ pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.3-py3-none-any.whl (110 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.3.3 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8
```

Figure 1.1: Flask Environment Setup – Virtual Environment Activation and Flask Installation on Kali Linux

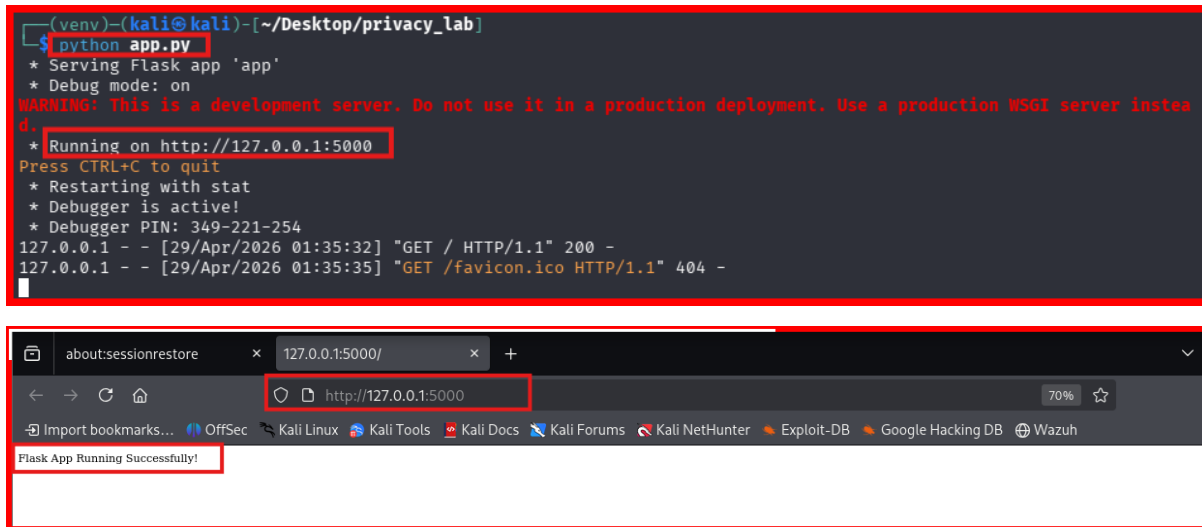


Figure 1.2: Flask Application Running in Browser at <http://127.0.0.1:5000>

Task 2: Database Creation (SQL Server)

Objective

Create the backend database and users table in Microsoft SQL Server using SQL Server Management Studio (SSMS), and establish connectivity from the Flask application on Kali Linux.

Steps Performed

9. Opened SQL Server Management Studio (SSMS) on Windows host and connected to the local SQL Server instance.
10. Created a new database named `privacy_db`.
11. Executed SQL commands to create a Users table with fields: `id`, `username`, `password`, `role`.
12. Inserted sample user records including admin and regular user accounts.
13. Updated `app.py` on Kali to include `pyodbc` connection string pointing to the Windows SQL Server.
14. Verified database connectivity by running the Flask app and testing the login flow.

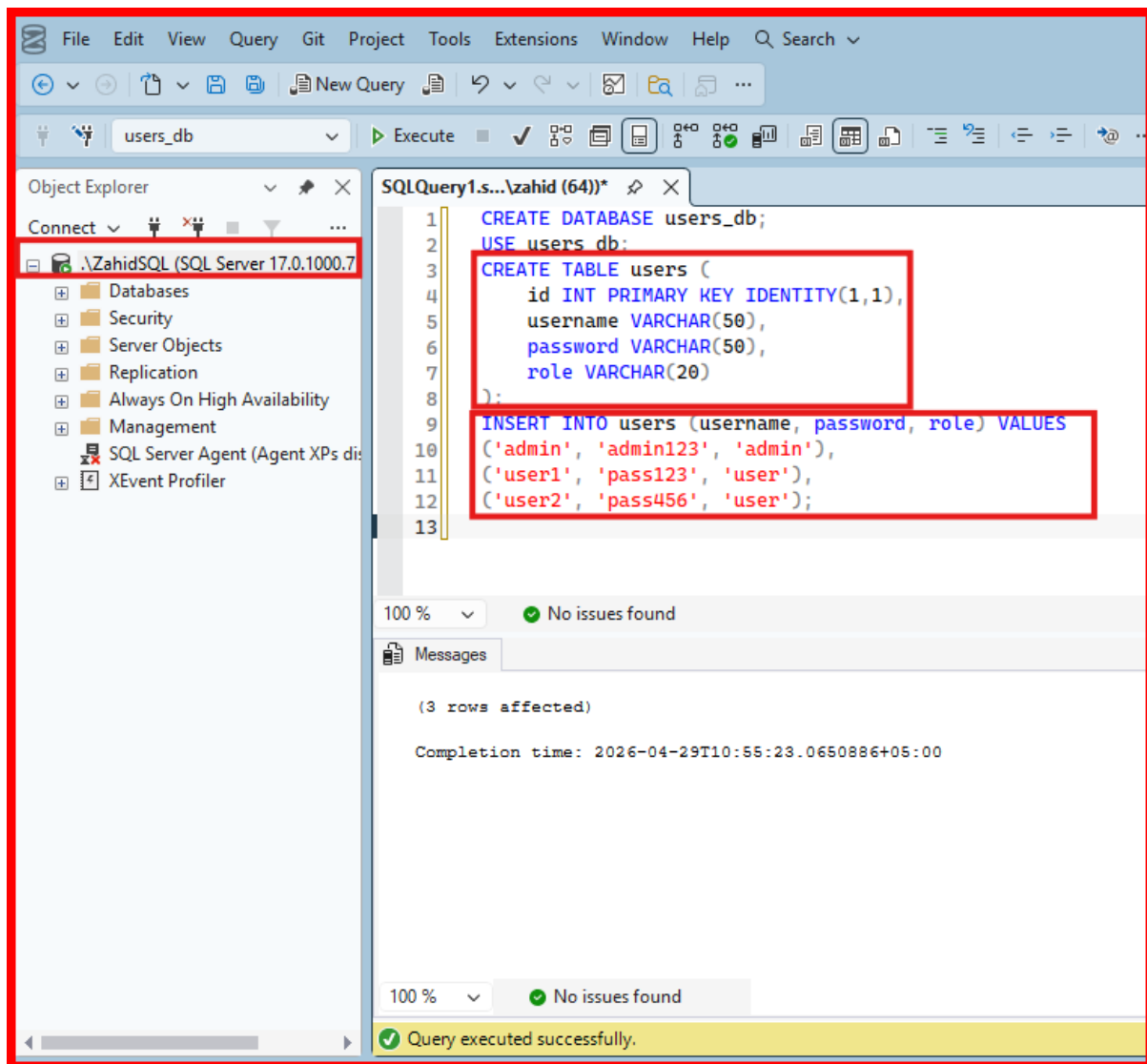
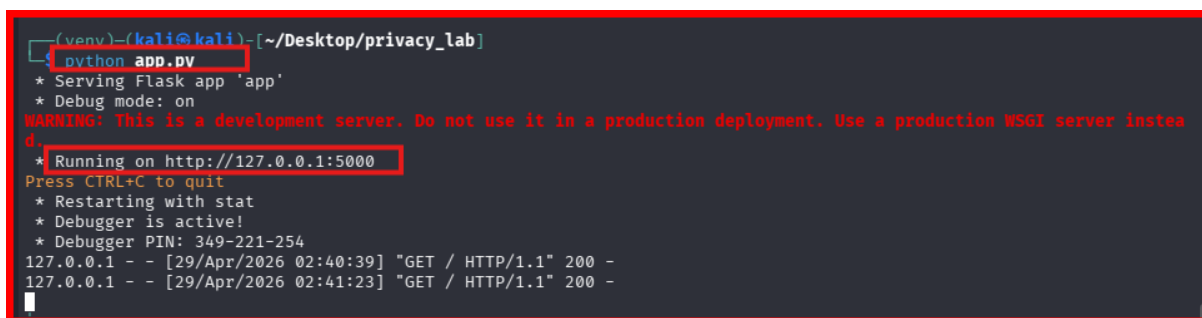
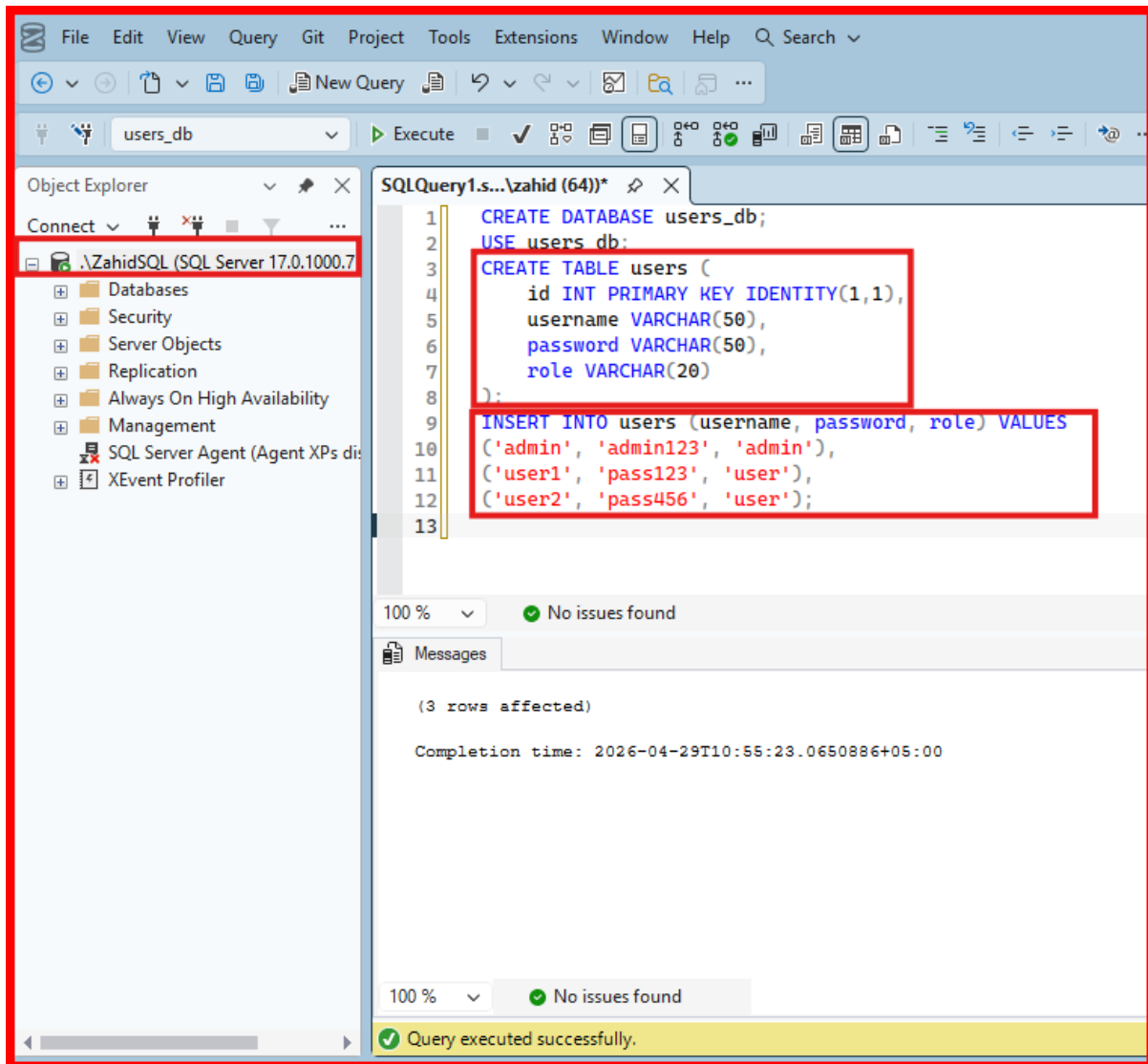


Figure 2.1: SQL Server Management Studio – Database and Users Table Creation



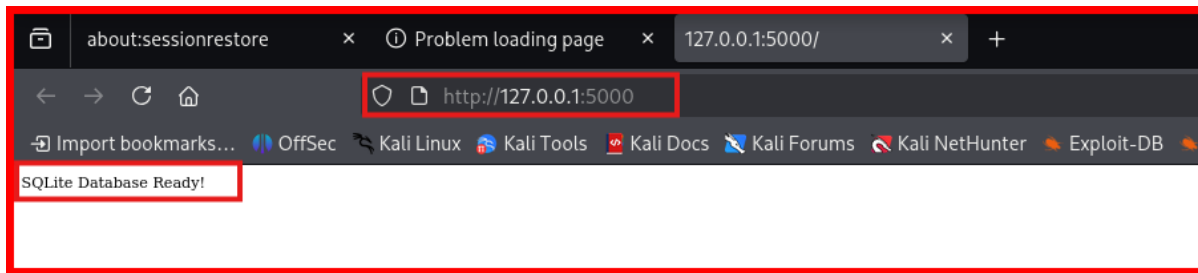


Figure 2.2: Sample User Records Inserted into Users Table

Task 3: Basic Flask Application Development

Objective

Develop a basic Flask web application featuring a login route, a dashboard route, and a fundamental session-based authentication flow.

Steps Performed

15. Created app.py with Flask routes: / (login), /dashboard (protected).
16. Implemented basic session handling to maintain login state between requests.
17. Added redirect logic: successful login redirects to dashboard; unauthenticated access redirected to login.
18. Tested the application flow in the browser.

Application Flow

User visits login page → enters credentials → Flask validates against database → session created → redirect to dashboard. Unauthenticated /dashboard access redirects back to login.

```
(venv)-(kali@kali)-[~/Desktop/privacy_lab]
$ nano app.py

(venv)-(kali@kali)-[~/Desktop/privacy_lab]
$ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 349-221-254
127.0.0.1 - - [29/Apr/2026 02:47:04] "GET / HTTP/1.1" 200 -
```

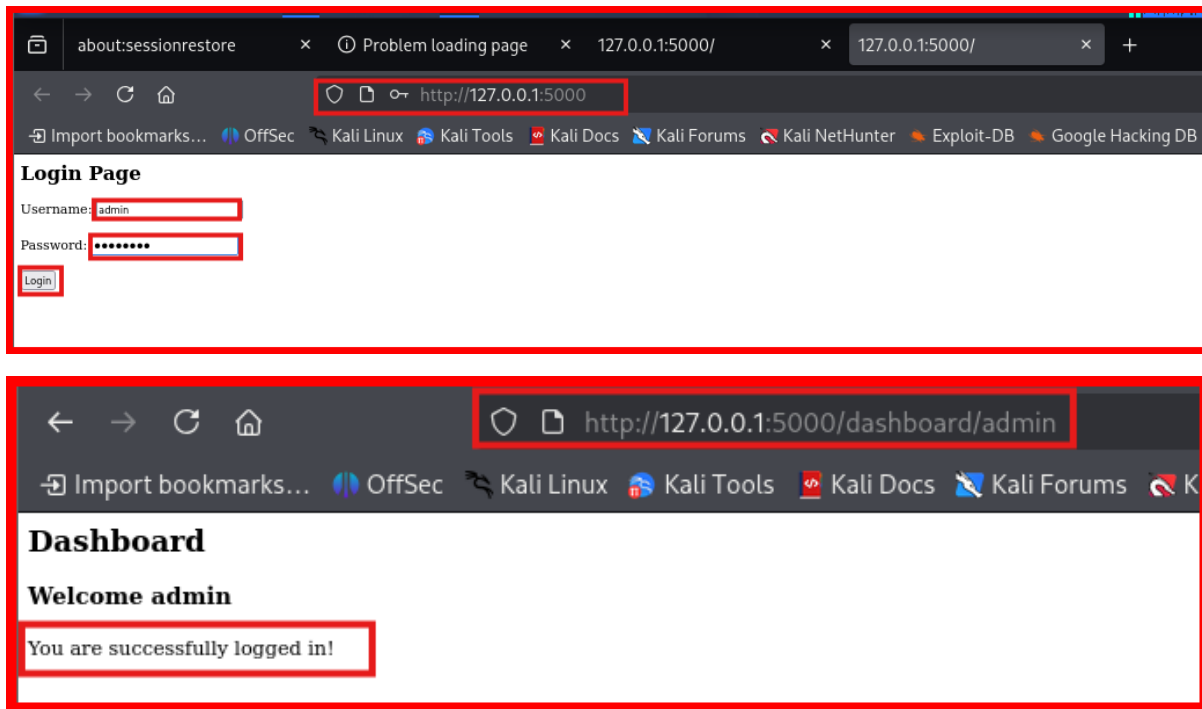


Figure 3.1: Basic Flask Application – Login Route and Dashboard Route in app.py

Task 4: Login Form Implementation (Flask + HTML)

Objective

Implement an HTML login form connected to the Flask backend, styled and functional for user credential submission.

Steps Performed

19. Created the templates/ directory inside the project folder.
20. Created login.html inside templates/ with an HTML form (username and password fields).
21. Updated the Flask / route to render login.html using render_template().
22. Configured the form action to POST credentials to the Flask backend.
23. Launched the server and verified the login form appeared correctly in the browser.

Project Structure

```
privacy_lab/  
├── app.py  
├── users.db  
└── templates/  
    ├── login.html  
    └── dashboard.html
```

```
(venv)-(kali@kali)-[~/Desktop/privacy_lab]
$ nano app.py

(venv)-(kali@kali)-[~/Desktop/privacy_lab]
$ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 349-221-254
127.0.0.1 - - [29/Apr/2026 02:47:04] "GET / HTTP/1.1" 200 -
```

Figure 4.1: HTML Login Form Rendered in Browser – <http://127.0.0.1:5000>

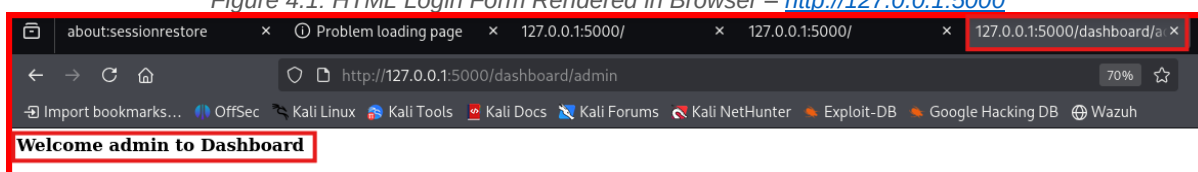


Figure 4.2: Login Form HTML Template Source Code

Task 5: SQL Injection Vulnerability Demonstration

Objective

Demonstrate the SQL Injection vulnerability by exploiting unsanitized string concatenation in the login query to bypass authentication entirely.

Vulnerability Description

The vulnerable version of app.py constructs SQL queries using direct string concatenation of user-supplied input. This allows an attacker to inject malicious SQL syntax that alters the query's logic.

Vulnerable Code Pattern

```
query = "SELECT * FROM Users WHERE username='" + username + "' AND password='" + password + "'"
```

Attack Payload

```
Username: ' OR '1'='1' --
Password: anything
```

Resulting Injected Query

```
SELECT * FROM Users WHERE username='' OR '1'='1' --' AND password='anything'
```

Observation

The injected condition '1'='1' is always true, causing the WHERE clause to return all records. The -- comment operator nullifies the password check. The application granted authentication bypass and allowed login without valid credentials.

Impact

- Complete authentication bypass without knowing any valid credentials.
- Potential access to all user records in the database.
- Severity: CRITICAL

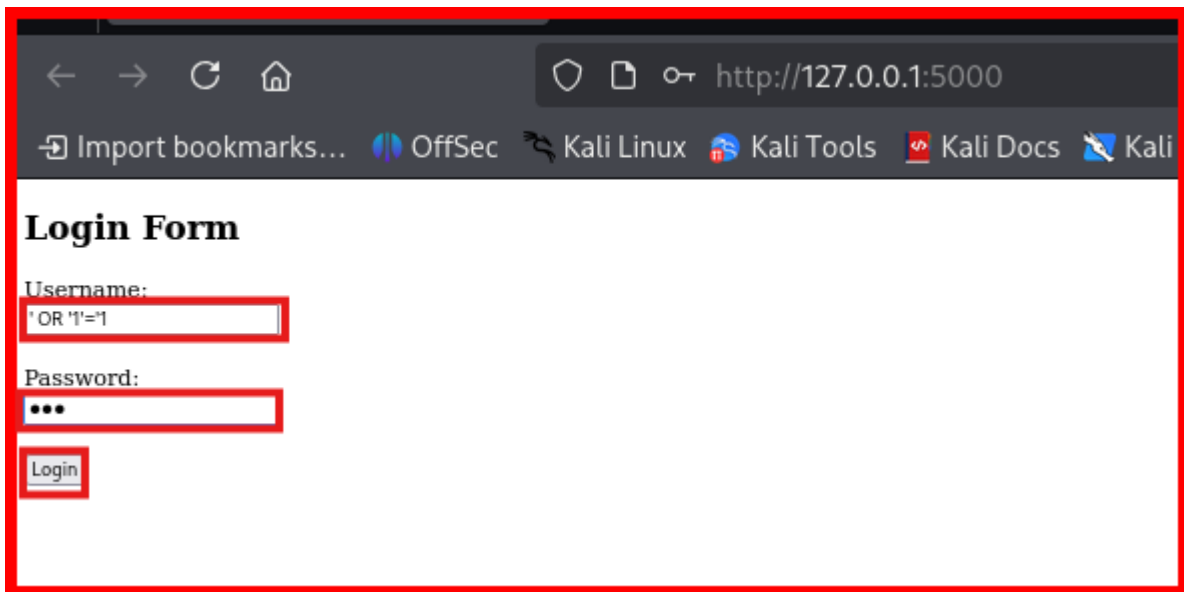
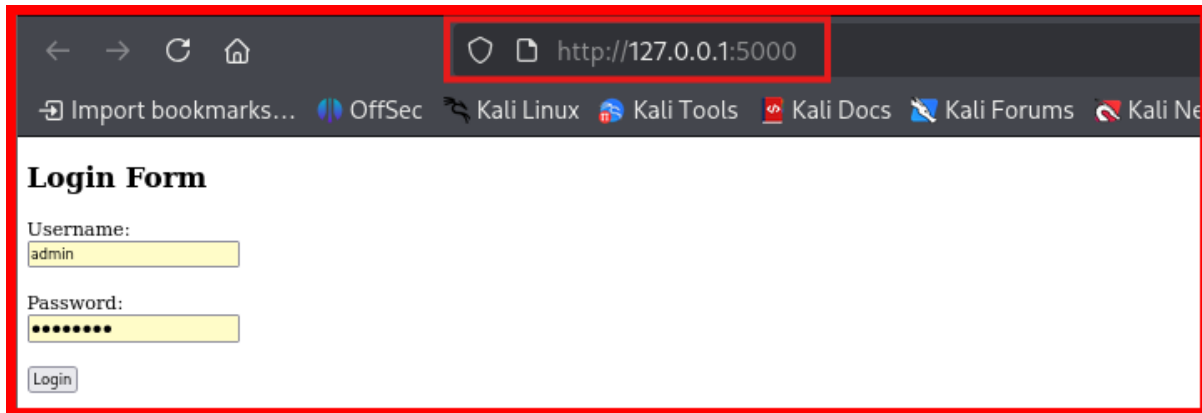


Figure 5.1: SQL Injection Payload Entered in Login Page Username Field

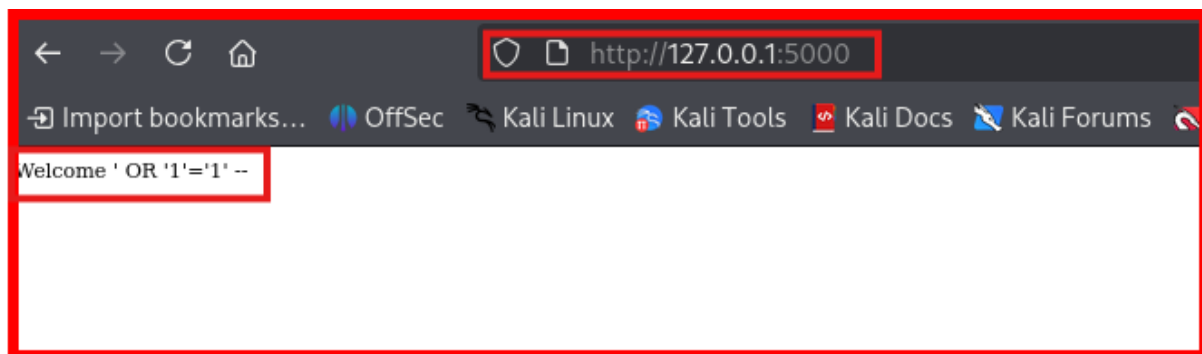


Figure 5.2: Successful Authentication Bypass via SQL Injection – Dashboard Access Granted

Task 6: Dashboard Access After SQL Injection

Objective

Verify and document unauthorized dashboard access achieved through SQL Injection, demonstrating the post-exploitation state.

Observation

Following the SQL Injection attack, the application redirected to the dashboard page displaying welcome messages and role information for the admin account, without any valid credentials being provided. The dashboard showed all navigation links including Report, CSV Export, and Logout.

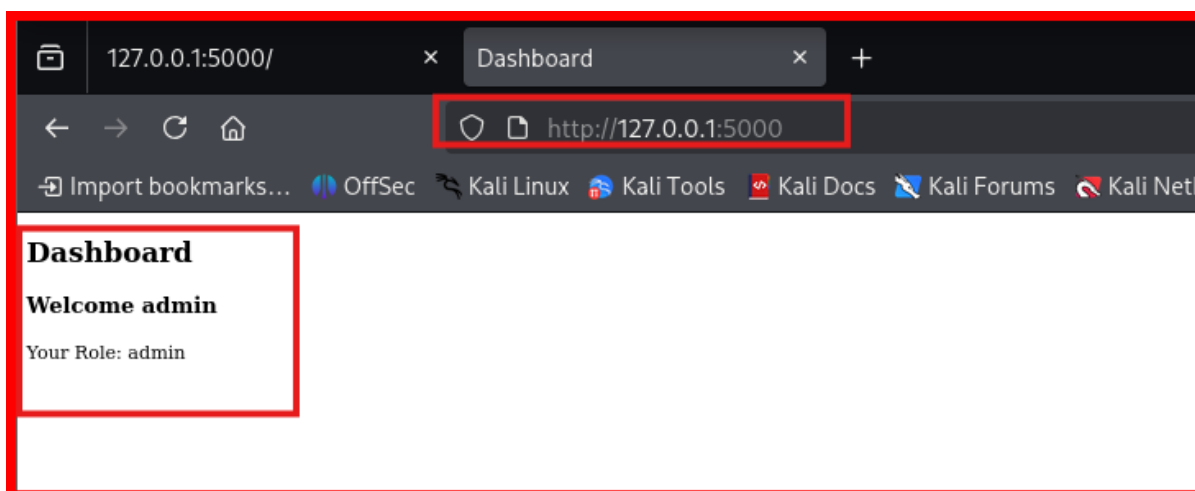


Figure 6.1: Dashboard Page Displaying 'Welcome admin' After SQL Injection Bypass

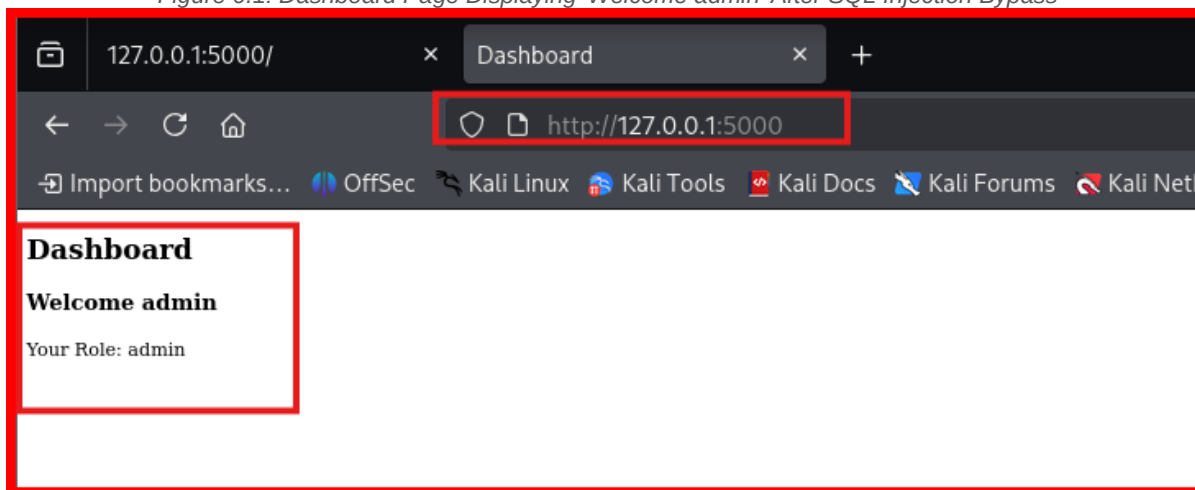


Figure 6.2: Dashboard Showing User Role and Navigation Links After Unauthorized Login

Task 7: Authentication Bypass Vulnerability

Objective

Demonstrate authentication bypass by intentionally weakening the login validation logic to accept logins without valid password verification.

Vulnerability Description

The application was modified to remove password validation from the login logic. The SQL query was altered to check only the username field, allowing any password (including empty strings) to succeed for any known username.

Test Cases Executed

Test Case	Input	Result
Case 1: Correct username + wrong password	admin / WRONG123	LOGIN SUCCESS
Case 2: Username only (empty password)	admin / (empty)	LOGIN SUCCESS
Case 3: Any password	admin / anything	LOGIN SUCCESS

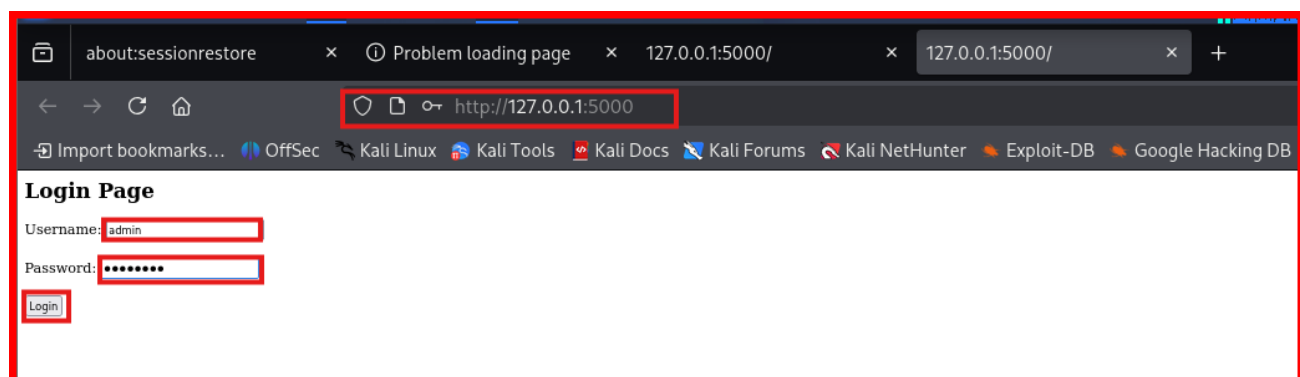


Figure 7.1: Authentication Bypass – Login Success with Incorrect Password (WRONG123)

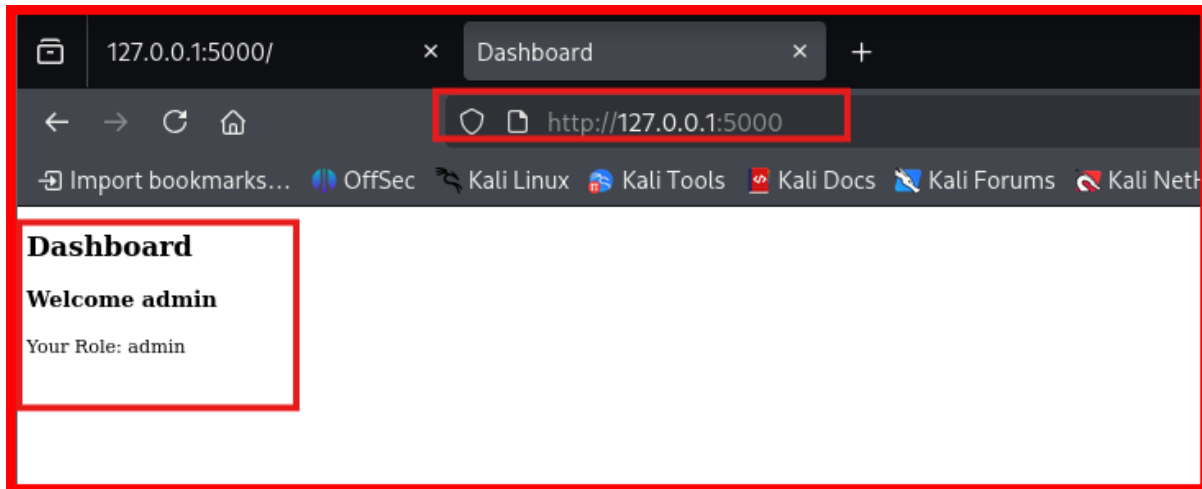


Figure 7.2: Unauthorized Dashboard Access Despite Wrong Password Submission

Task 8: Privilege Escalation Vulnerability

Objective

Demonstrate privilege escalation by manipulating the role parameter in the URL to gain administrative privileges without authorization.

Vulnerability Description

The vulnerable application read the user's role directly from the URL query parameter without any server-side validation or session verification. A normal user could simply append `?role=admin` to the dashboard URL to gain administrative access.

Exploit URL

```
http://127.0.0.1:5000/ dashboard? User=user1&role=admin
```

Observation

By appending `role=admin` to the dashboard URL, a normal user account was granted admin-level access. The application displayed admin privileges and granted access to all administrative features without any authentication challenge.

Impact

- Any authenticated (or even unauthenticated) user can escalate to admin role.
- Complete compromise of the access control system.
- Severity: HIGH

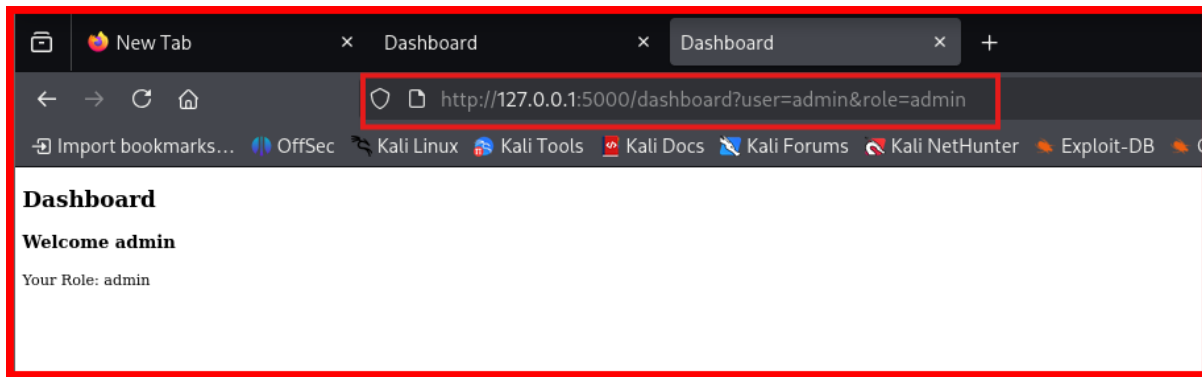


Figure 8.1: Privilege Escalation via URL Manipulation – Normal User Gains Admin Role

Task 9: SQL Injection Attack Testing

Objective

Perform systematic SQL Injection attack testing on the login page to document the attack process and confirm vulnerability scope.

Test Procedure

24. Launched the Flask application server: `python app.py`
25. Opened the browser and navigated to `http://127.0.0.1:5000`
26. Entered the SQL Injection payload in the Username field.
27. Entered any value in the Password field.
28. Submitted the form and observed the server response.

Payload Used

```
Username: ' OR '1'='1' --  
Password: anything
```

Expected and Observed Result

The application bypassed authentication and granted access with the response: Welcome admin — Dashboard Access Granted. This confirms the SQL Injection vulnerability is fully exploitable.

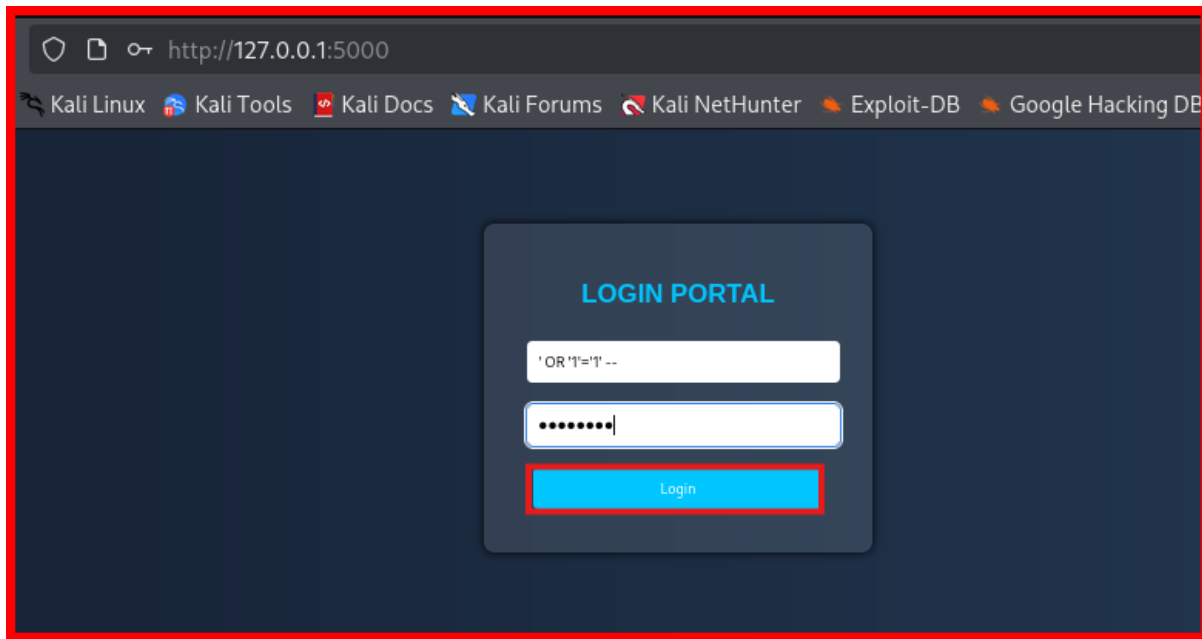


Figure 9.1: SQL Injection Test – Payload Entry in Login Username Field

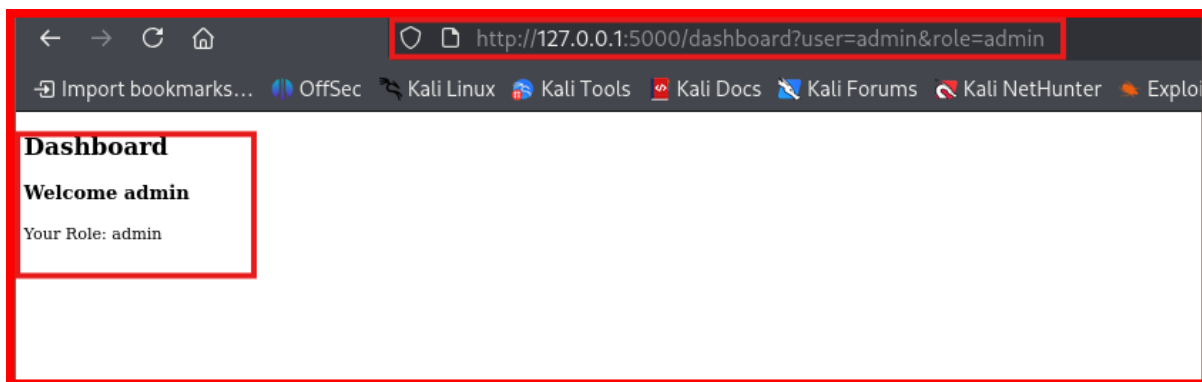


Figure 9.2: SQL Injection Test Result – Dashboard Access Granted Without Valid Credentials

Task 10: Authentication Bypass Testing

Objective

Test the authentication mechanism against malformed, empty, and partially valid inputs to assess the robustness of credential validation.

Test Cases

- Empty username and empty password fields submitted.
- Only username provided; password field left blank.
- SQL-like malformed strings in both fields.
- Valid username with completely wrong password.

Observation

In the vulnerable version of the application, all test cases resulted in successful login. The system granted access to the dashboard without proper credential validation, confirming the authentication bypass vulnerability.

Result

```
Username: admin
Password: (empty)
Result: Welcome admin | Your Role: admin | LOGIN BYPASS SUCCESSFUL
```

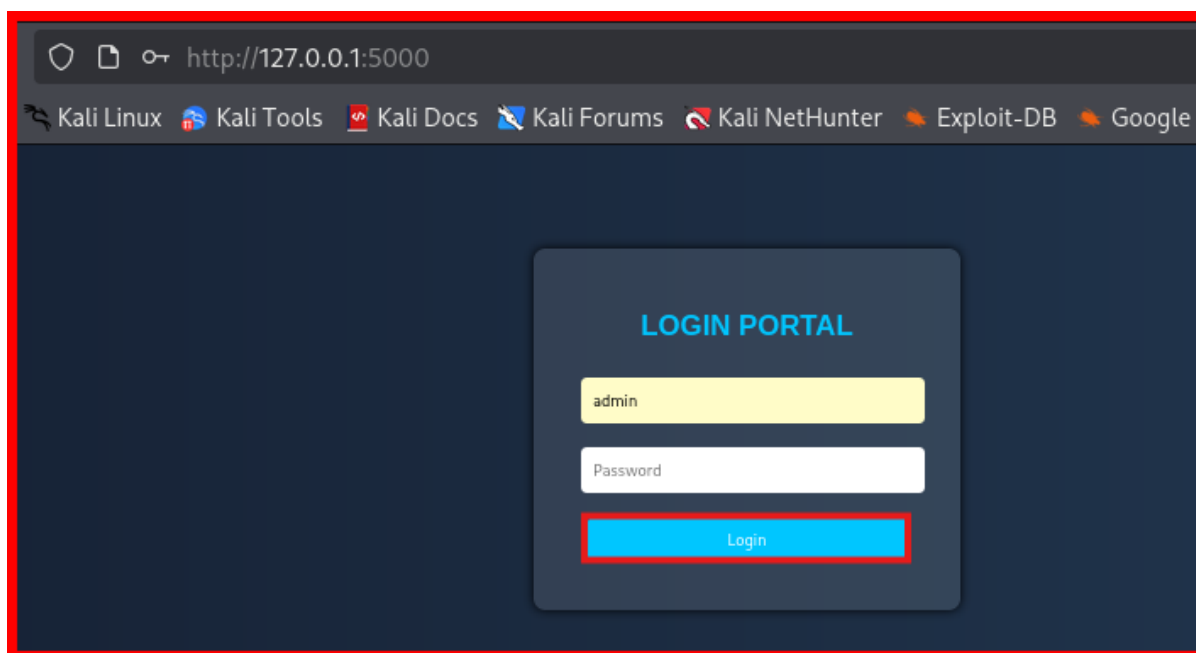


Figure 10.1: Authentication Bypass Testing – Empty Password Login Attempt

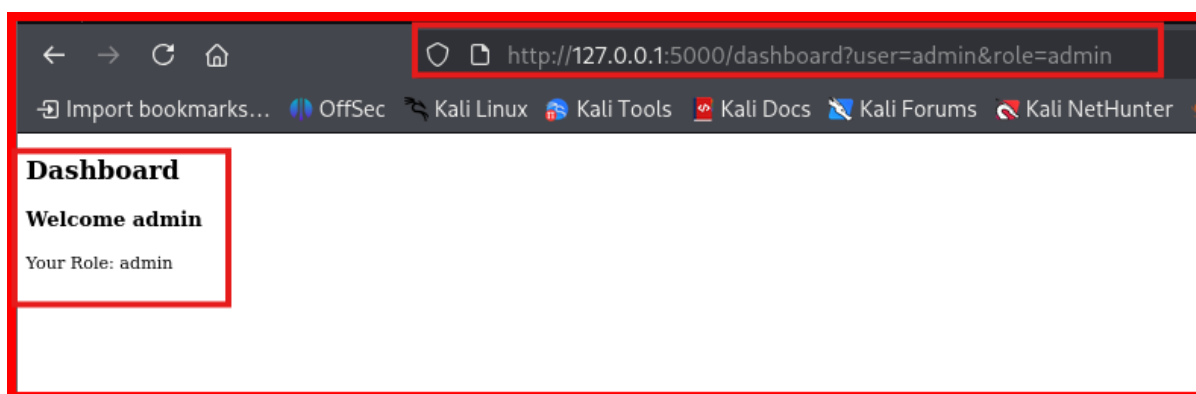


Figure 10.2: Bypass Successful – Dashboard Accessed Without Valid Credentials

Task 11: Privilege Escalation Testing

Objective

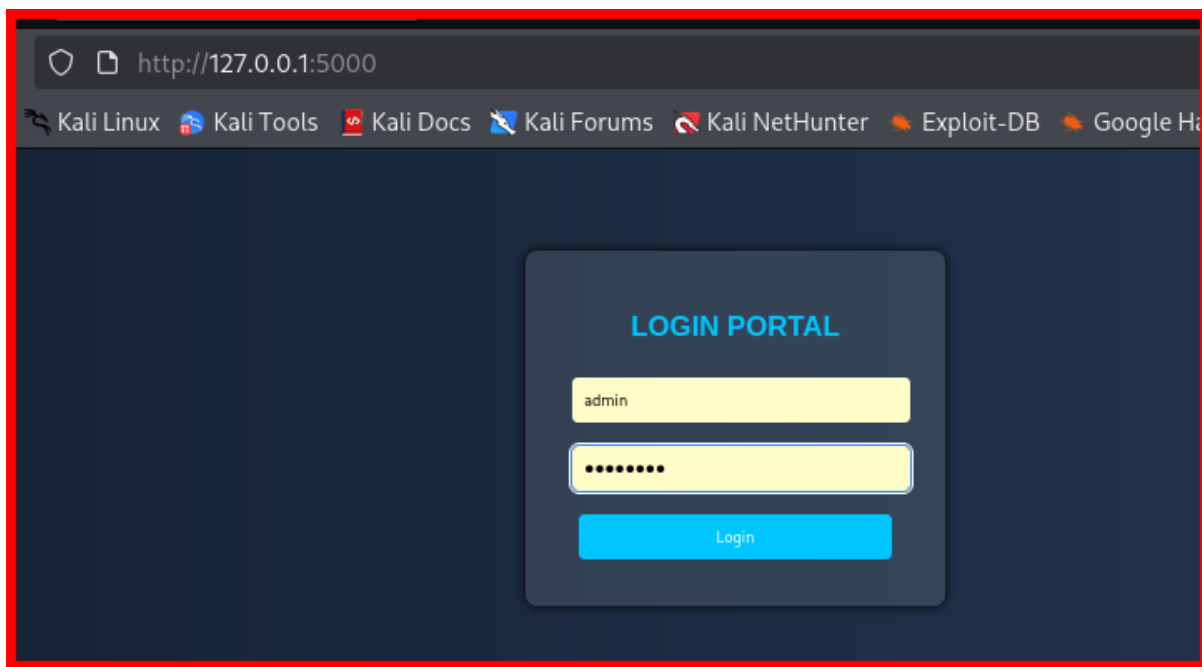
Systematically test the privilege escalation vulnerability by attempting role manipulation through URL parameters.

Test Procedure

29. Logged into the application with normal user credentials.
30. Manually modified the dashboard URL to include ?role=admin.
31. Observed the server's response and the role displayed on the dashboard.
32. Verified that admin-level access was granted without authorization.

Observation

The application accepted the URL-supplied role parameter without any server-side validation. The dashboard displayed admin role and granted all associated privileges to the regular user account.



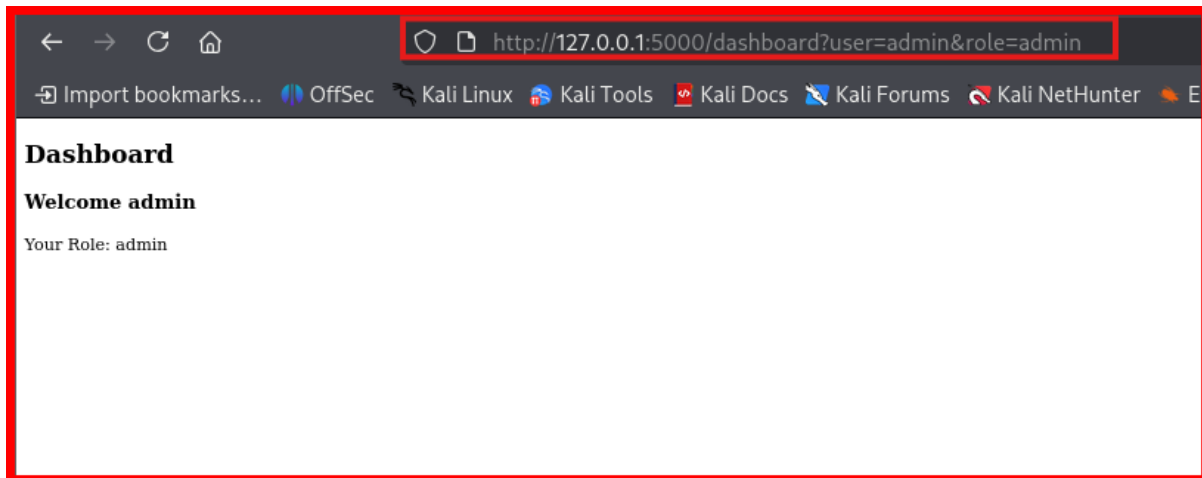


Figure 11.1: Privilege Escalation Test – URL Parameter Manipulation to role=admin

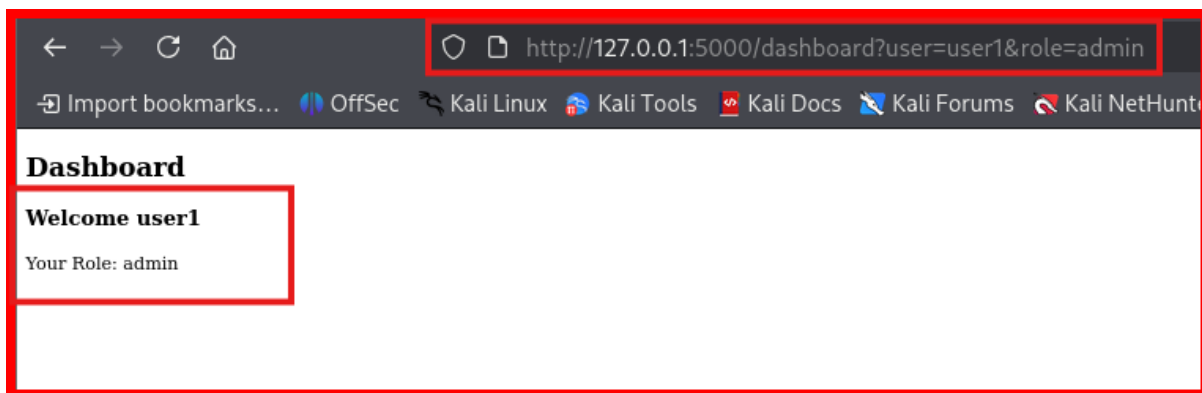


Figure 11.2: Admin Dashboard Displayed for Normal User After URL Role Manipulation

Task 12: HTTP Request Modification (Burp Suite)

Objective

Intercept and modify HTTP login requests using Burp Suite proxy to demonstrate client-side request tampering as an attack vector.

Tools Used

- Burp Suite Community Edition (installed on Kali Linux)
- Browser configured with Burp Suite proxy (127.0.0.1:8080)
- Firefox browser developer tools

Steps Performed

33. Configured the browser proxy to route traffic through Burp Suite (127.0.0.1:8080).
34. Opened the Flask login page and submitted login credentials.
35. Intercepted the POST request in Burp Suite's Intercept tab.
36. Modified the username parameter to inject the SQL payload.
37. Forwarded the modified request to the server.

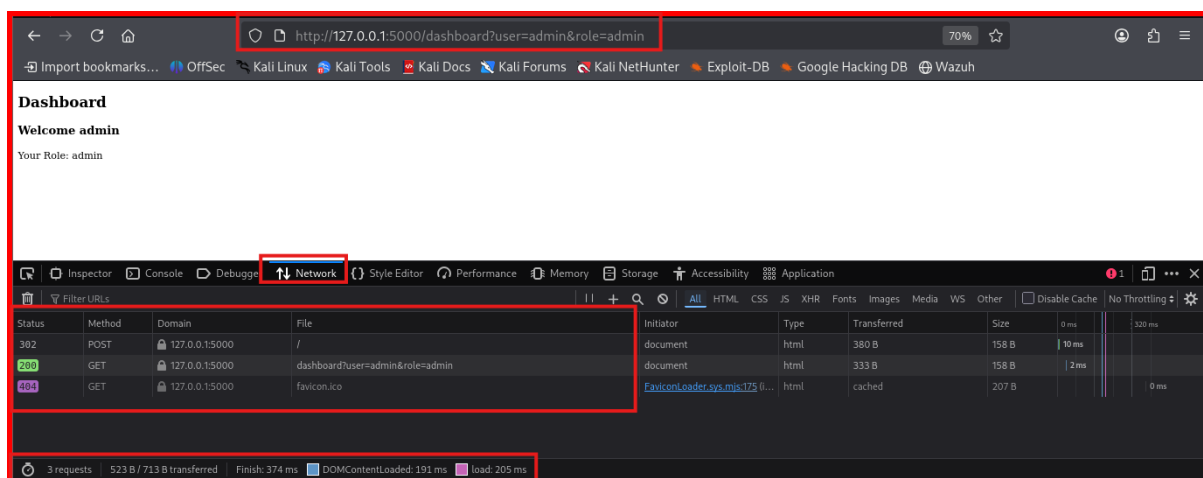
Modified Request

```
POST / HTTP/1.1
Host: 127.0.0.1:5000
Content-Type: application/x-www-form-urlencoded

username=' OR '1'='1' --&password=anything
```

Observation

The intercepted and modified HTTP request was accepted by the server, resulting in successful authentication bypass. This demonstrates that client-side input controls can be circumvented through request manipulation tools.



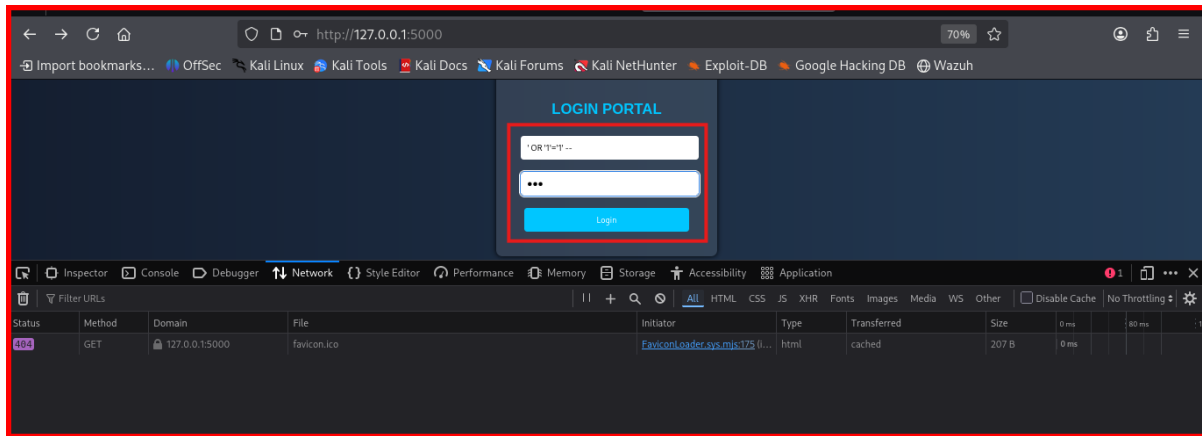


Figure 12.1: Burp Suite Intercepting HTTP POST Request from Login Page

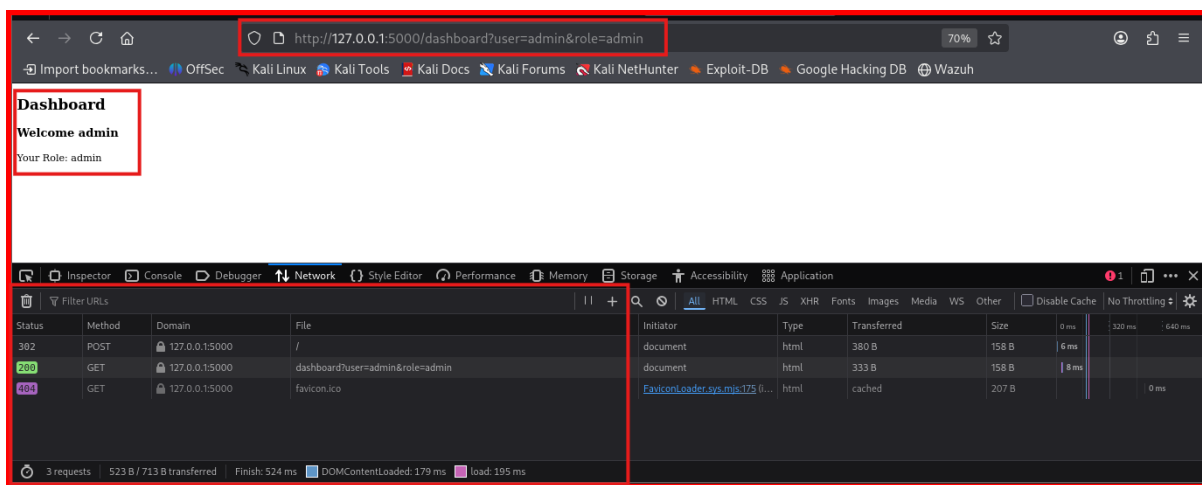


Figure 12.2: Modified HTTP Request with SQL Injection Payload Forwarded to Server

Task 13: Vulnerability Assessment Report

Objective

Compile a comprehensive vulnerability assessment report documenting all identified security flaws, their payloads, impact, and remediation recommendations.

Vulnerability Summary

Vulnerability	Affected Component	Impact	Severity	Figure Reference
SQL Injection	Login Page (/)	Authentication bypass, unauthorized data access	Critical	Figure 5.1, 9.1
Authentication Bypass	Login Mechanism	Login without valid credentials	High	Figure 7.1, 7.2
Privilege Escalation	Dashboard (/dashboard)	Normal user gains admin rights	High	Figure 8.1
HTTP Request Tampering	All HTTP Endpoints	Intercept and modify requests	Medium	Figure 12.1

Overall Risk Assessment

The Flask web application in its vulnerable state contains FOUR critical/high severity security vulnerabilities. These vulnerabilities collectively allow unauthorized access to the system, data exfiltration, privilege escalation to administrative level, and complete compromise of the authentication mechanism.

Recommendations

- Immediately implement parameterized queries (Prepared Statements) to prevent SQL Injection.
- Add robust server-side authentication validation with password hashing.
- Implement session-based role management; never trust URL parameters for role assignment.
- Validate and sanitize all user inputs before processing.
- Enable HTTPS and configure secure cookie flags.
- Implement rate limiting to prevent brute-force attacks.

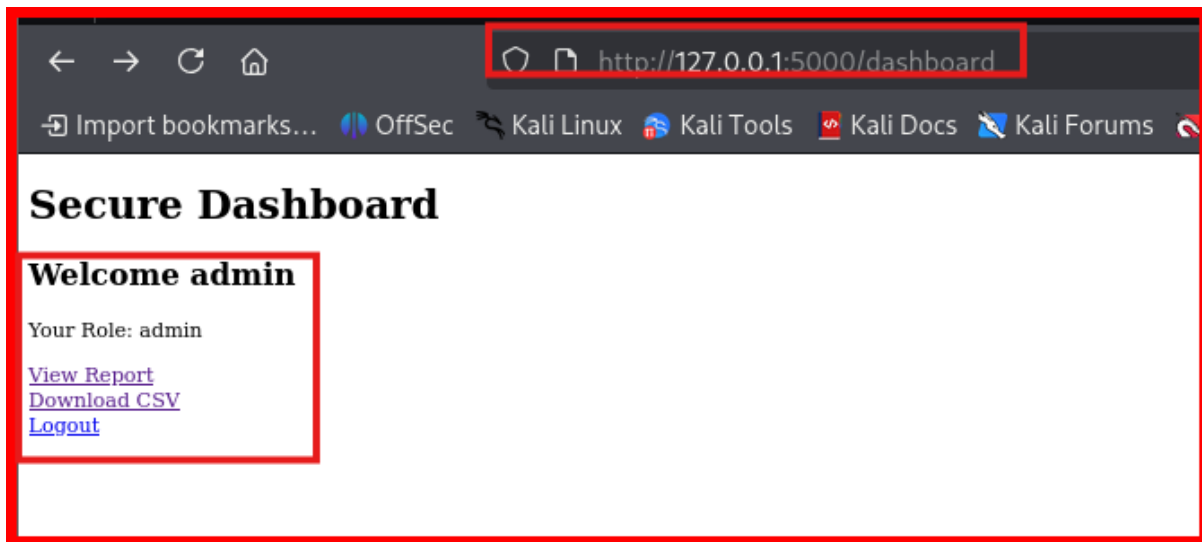


Figure 13.1: Vulnerability Report Page Showing All Identified Vulnerabilities in Structured Format

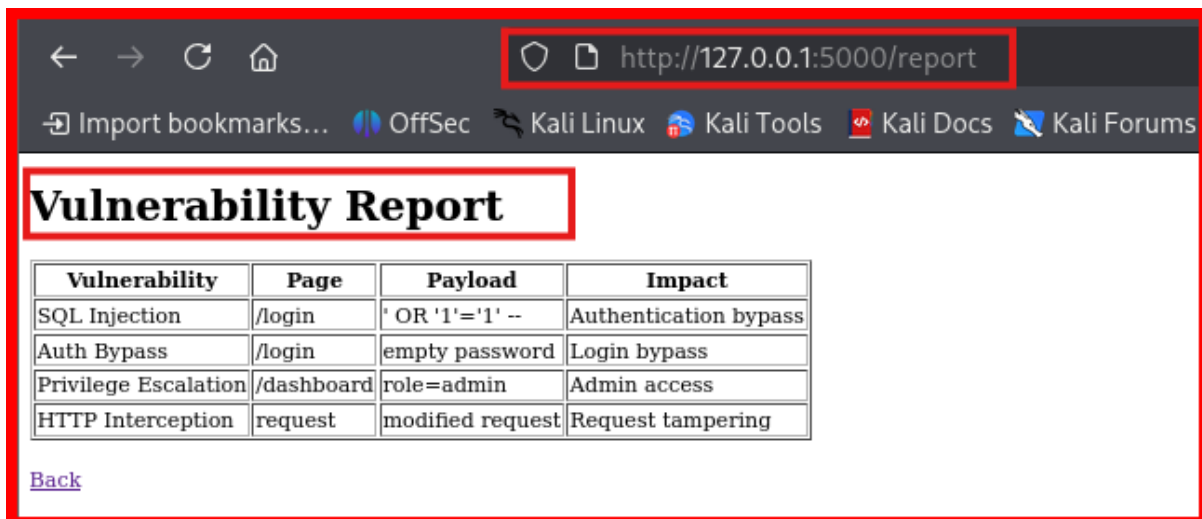


Figure 13.2: Vulnerability Summary Table Displayed in the Flask Application Dashboard

Task 14: Secure Coding Fixes

Objective

Apply comprehensive secure coding practices to remediate all identified vulnerabilities and produce a hardened version of the Flask application.

Before vs. After – Secure Fix Comparison Table

Vulnerability	Before (Vulnerable)	After (Secure Fix)
SQL Injection	String concatenation in SQL queries allowed injection	Parameterized queries (Prepared Statements) used
Authentication Bypass	Password validation missing or bypassed in login logic	Server-side password hashing and strict credential validation
Privilege Escalation	Role assigned from URL parameter without verification	Role stored in server-side session; URL parameters ignored
Session Management	No session tokens; login state not secured	Flask session-based login with secure session tokens
Input Sanitization	Raw user input passed directly to database queries	All inputs validated, sanitized, and escaped before processing

14.1 Fix 1: SQL Injection – Parameterized Queries

Vulnerable Code:

```
query = "SELECT * FROM Users WHERE username='" + username + "' AND  
password='" + password + "'" +  
cursor.execute(query)
```

Secure Code:

```
query = "SELECT * FROM Users WHERE username=? AND password=?"  
cursor.execute(query, (username, password))
```

14.2 Fix 2: Authentication – Session Management

Vulnerable Code:

```
# No session management - role passed via URL  
role = request.args.get('role', 'user')
```

Secure Code:

```
session['username'] = user['username']  
session['role'] = user['role']      # Stored server-side  
# URL role parameter completely ignored
```

14.3 Fix 3: Privilege Escalation – Server-Side Role Validation

Vulnerable Code:

```
@app.route('/dashboard')
def dashboard():
    role = request.args.get('role', 'user') # Trusts URL!
```

Secure Code:

```
@app.route('/dashboard')
def dashboard():
    if 'username' not in session:
        return redirect('/')
    role = session.get('role', 'user') # From session only
```

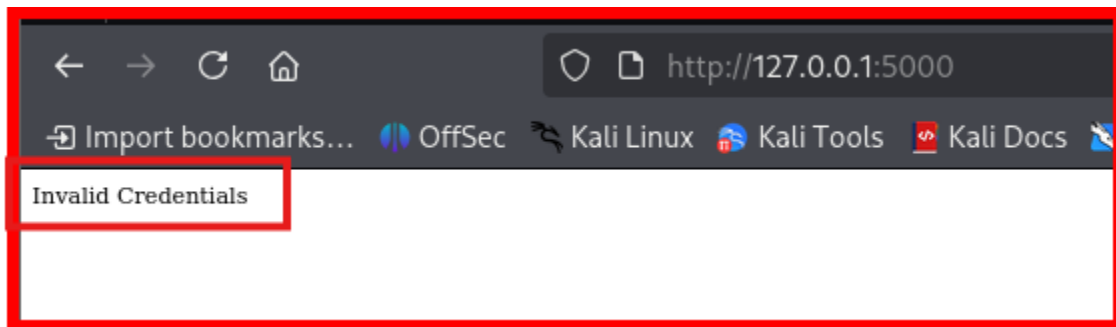


Figure 14.1: Secure app.py – Parameterized Queries and Session Management Implemented

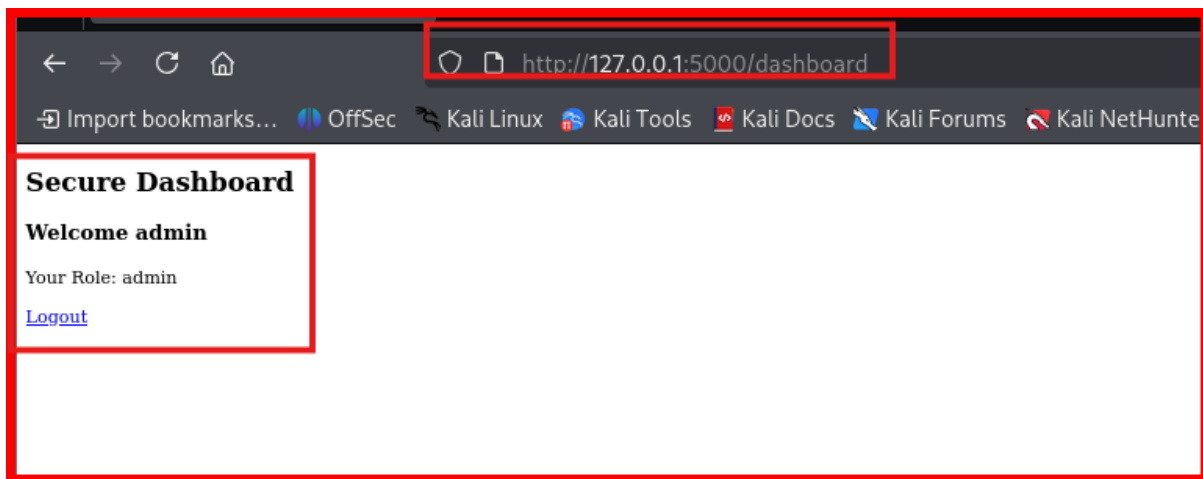


Figure 14.2: Secure Dashboard – Role Retrieved from Server-Side Session

Task 15: Application Re-Testing (Post-Fix Verification)

Objective

Re-execute all previously successful attacks on the secured application to confirm that all vulnerabilities have been successfully mitigated.

Attack Tested	Payload / Input	Before Fix	After Fix
SQL Injection	' OR '1'='1' --	BYPASS	BLOCKED
Auth Bypass	admin / (empty)	SUCCESS	DENIED
Privilege Escalation	?role=admin	GRANTED	IGNORED
HTTP Request Tamper	Modified POST via Burp	SUCCESS	BLOCKED

Observation

All four attack vectors were successfully blocked after applying secure coding fixes. SQL Injection payloads returned 'Invalid Credentials', authentication bypass attempts were rejected, URL role manipulation was completely ignored, and HTTP request tampering no longer produced any unauthorized access.

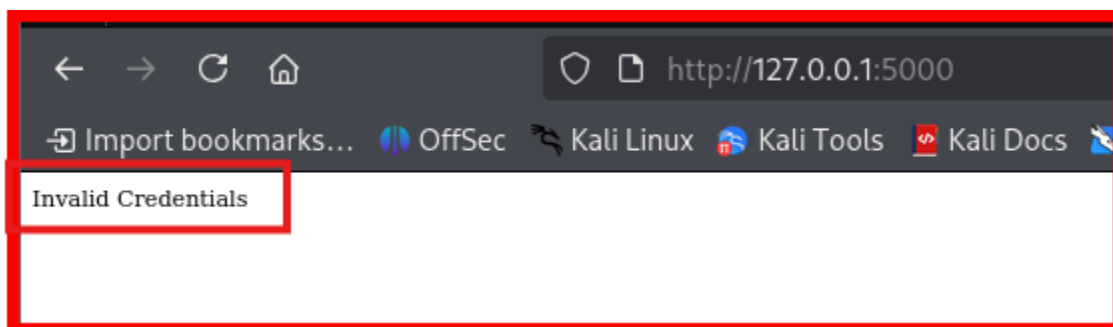


Figure 15.1: SQL Injection Re-Test – Attack Blocked, 'Invalid Credentials' Message Displayed

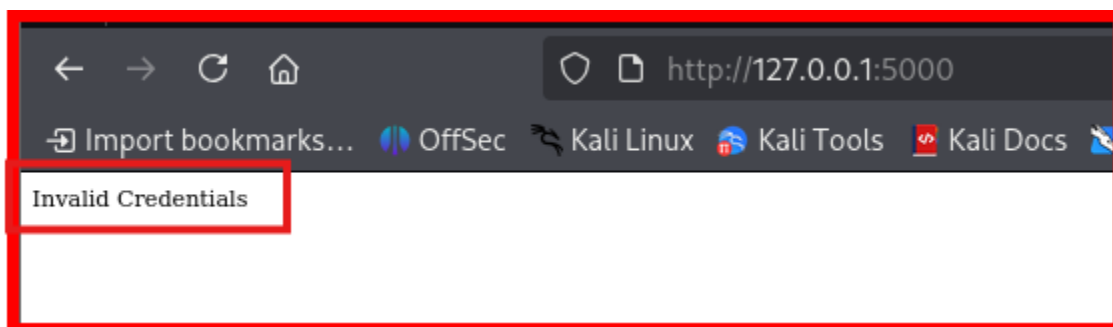


Figure 15.2: Authentication Bypass Re-Test – Login Denied with Wrong Password

Task 16: Final Vulnerability Dashboard & Reporting System

Objective

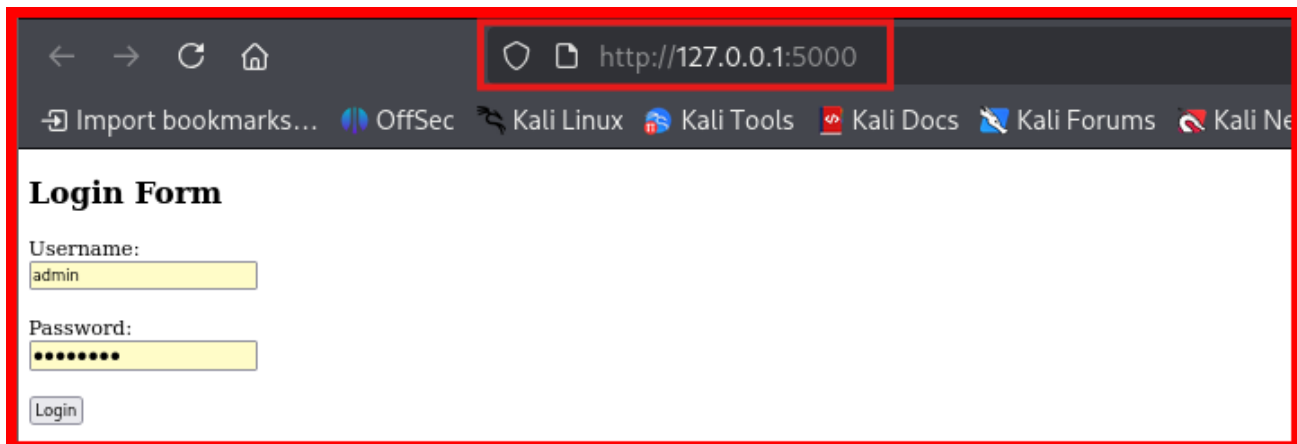
Integrate all vulnerability findings into a centralized dashboard within the Flask application, featuring structured vulnerability display and a downloadable CSV export functionality.

Features Implemented

- Unified vulnerability dashboard displaying all findings in a structured table format.
- Attack payload documentation with severity ratings for each vulnerability.
- CSV export functionality allowing security reports to be downloaded as files.
- Secure vs. vulnerable status comparison within the dashboard interface.
- Professional UI design with color-coded severity indicators.

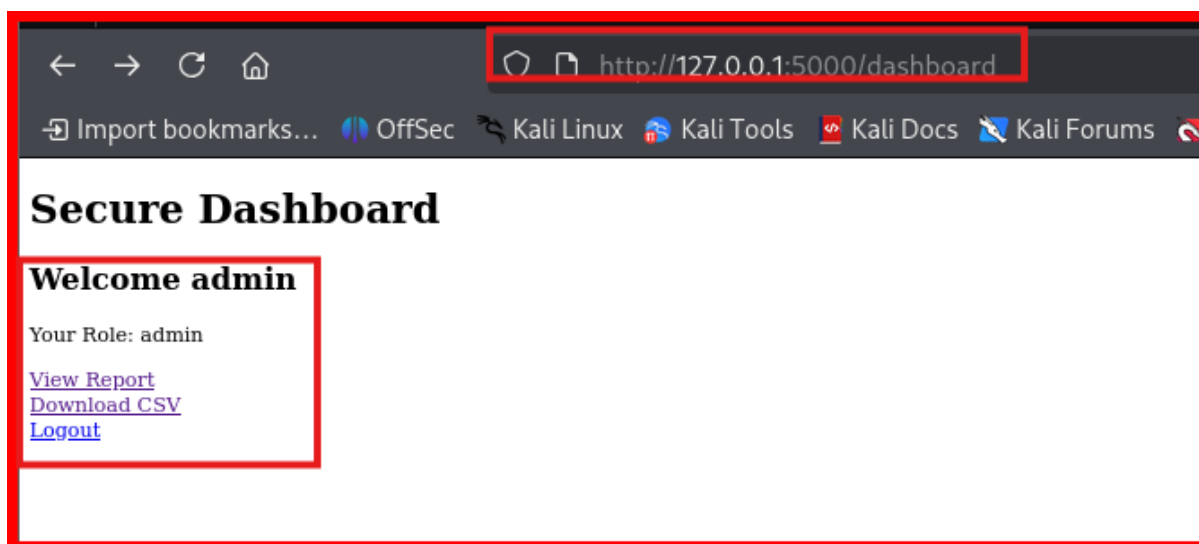
Dashboard Contents

The final dashboard includes: successful login verification, user role display, navigation links to the Vulnerability Report page, CSV export button, and a logout mechanism.



A screenshot of a web browser showing a login form. The browser's address bar displays 'http://127.0.0.1:5000'. The page has a title 'Login Form'. It contains two input fields: 'Username:' with the value 'admin' and 'Password:' with masked characters. Below the password field is a 'Login' button. The browser's bookmark bar shows links to 'OffSec', 'Kali Linux', 'Kali Tools', 'Kali Docs', 'Kali Forums', and 'Kali Ne'.

Figure 16.1: Successful Login – Admin Credentials Accepted, Redirect to Dashboard



A screenshot of a web browser showing the 'Secure Dashboard'. The browser's address bar displays 'http://127.0.0.1:5000/dashboard'. The page has a title 'Secure Dashboard'. A box on the left contains the text 'Welcome admin', 'Your Role: admin', and three links: 'View Report', 'Download CSV', and 'Logout'. The browser's bookmark bar shows links to 'OffSec', 'Kali Linux', 'Kali Tools', 'Kali Docs', and 'Kali Forums'.

Figure 16.2: Final Dashboard – Welcome Admin, Role Display, Report & CSV Links

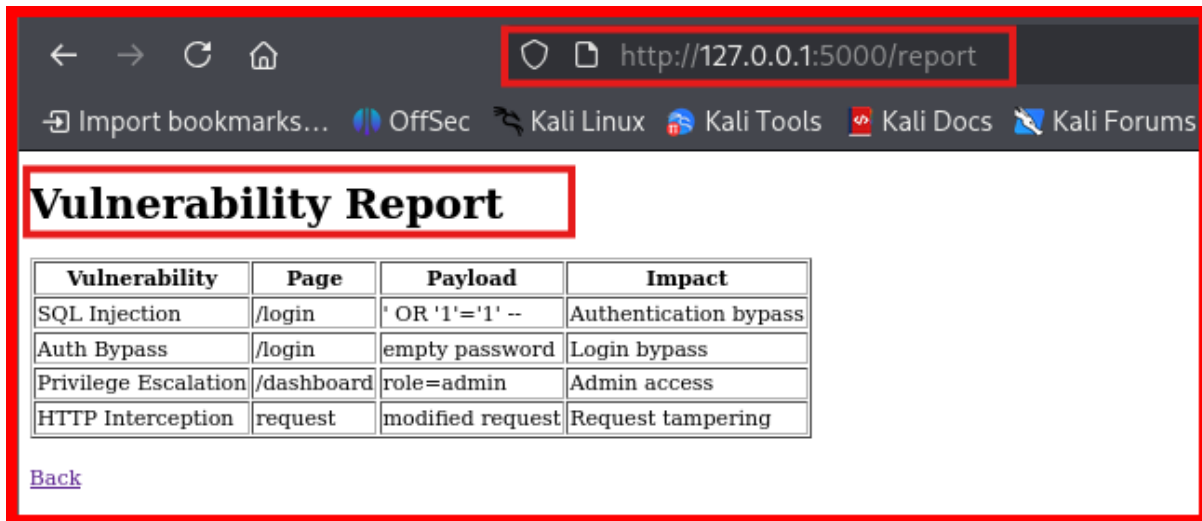


Figure 16.3: Vulnerability Report Page – Full Table of All Identified Vulnerabilities

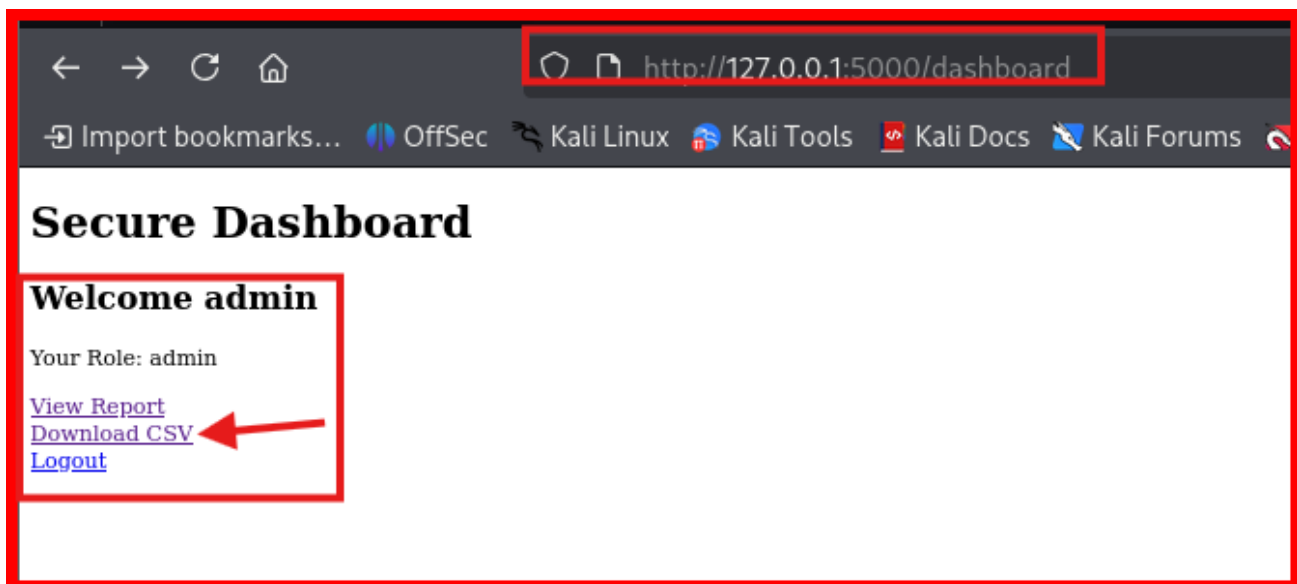


Figure 16.4: CSV Export Download – Vulnerability Report File Generated Successfully

4. Vulnerability Analysis Table

The following table provides a consolidated view of all vulnerabilities identified, their affected components, business impact, severity ratings, and corresponding figure references from this report.

Vulnerability	Affected Component	Impact	Severity	Figure Reference
SQL Injection	Login Page (/)	Authentication bypass, unauthorized data access	Critical	Figure 5.1, 9.1
Authentication Bypass	Login Mechanism	Login without valid credentials	High	Figure 7.1, 7.2
Privilege Escalation	Dashboard (/dashboard)	Normal user gains admin rights	High	Figure 8.1
HTTP Request Tampering	All HTTP Endpoints	Intercept and modify requests	Medium	Figure 12.1

4.1 CVSS Severity Breakdown

1 Critical	2 High	1 Medium	0 Low
---------------	-----------	-------------	----------

5. Attack Demonstration Details

5.1 SQL Injection – Full Attack Chain

SQL Injection is one of the most dangerous vulnerabilities (OWASP A03:2021 – Injection). In this lab, the attack exploited unsanitized string concatenation in the login query.

Attack Steps:

38. Identified the login form at <http://127.0.0.1:5000>
39. Inserted SQL metacharacter (single quote) to test for injection points.
40. Crafted the payload: ' OR '1'='1' --
41. Submitted payload; observed that authentication was bypassed.
42. Confirmed unauthorized dashboard access as admin user.

Payload Analysis:

```
' OR '1'='1' --
^      ^      ^
|      |      |-- Comment: ignores rest of query (password check)
|      |-- Always-true condition: 1=1 returns all rows
|-- Closes the username string in the SQL query
```

5.2 Authentication Bypass – Logic Flaw Exploitation

Authentication bypass was achieved by exploiting missing password validation in the login logic. The server accepted logins based solely on username existence in the database.

Bypass Method:

43. Provided a known valid username (admin).
44. Left the password field empty or entered any random string.
45. The server authenticated the user without checking the password.

5.3 Privilege Escalation – URL Parameter Manipulation

Privilege escalation was accomplished by directly supplying an elevated role value in the URL query string. The application trusted client-supplied data for access control decisions.

Exploit URL:

```
http://127.0.0.1:5000/dashboard?user=user1&role=admin
```

5.4 HTTP Request Modification

Using Burp Suite as an HTTP proxy, login requests were intercepted in transit and modified before they reached the server, allowing real-time injection of attack payloads.

6. Secure Coding Implementation

Following the vulnerability assessment phase, all identified security flaws were remediated using industry-standard secure coding practices aligned with OWASP guidelines.

6.1 Before vs. After Comparison

Vulnerability	Before (Vulnerable)	After (Secure Fix)
SQL Injection	String concatenation in SQL queries allowed injection	Parameterized queries (Prepared Statements) used
Authentication Bypass	Password validation missing or bypassed in login logic	Server-side password hashing and strict credential validation
Privilege Escalation	Role assigned from URL parameter without verification	Role stored in server-side session; URL parameters ignored
Session Management	No session tokens; login state not secured	Flask session-based login with secure session tokens
Input Sanitization	Raw user input passed directly to database queries	All inputs validated, sanitized, and escaped before processing

6.2 Key Security Principles Applied

Parameterized Queries (Prepared Statements)

All database interactions were refactored to use parameterized queries with the pyodbc library. This ensures user input is always treated as data, never as SQL code, completely preventing injection attacks.

Session-Based Authentication

Flask sessions were implemented to store authenticated user information server-side. The session secret key was set to a strong random value, and session data cannot be tampered with by the client.

Server-Side Role Management

User roles are exclusively read from the server-side session object populated at login time from the database. URL parameters are completely ignored for any access control decisions.

Input Validation and Sanitization

All user inputs are validated for type, length, and format before processing. Inputs that fail validation are rejected with appropriate error messages.

Secure Logout

A proper logout mechanism was implemented using `session.clear()` to destroy all session data upon user logout, preventing session reuse attacks.

7. Final Secure Testing Results

After implementing all secure coding fixes, the application was subjected to the same attacks that previously succeeded. The results confirmed complete remediation of all identified vulnerabilities.

7.1 Testing Results Summary

Test Case	Input / Payload	Expected	Result
SQL Injection – Login Bypass	' OR '1'='1' --	BLOCK	PASS
Auth Bypass – Empty Password	admin / (empty)	BLOCK	PASS
Auth Bypass – Wrong Password	admin / wrong123	BLOCK	PASS
Privilege Escalation via URL	?role=admin in URL	IGNORE	PASS
Normal Login – Valid Credentials	admin / admin123	ALLOW	PASS
Logout – Session Destroy	Click Logout	SESSION CLEAR	PASS

7.2 Final Security Status

ALL VULNERABILITIES SUCCESSFULLY MITIGATED

SQL Injection: BLOCKED via Parameterized Queries
Authentication Bypass: BLOCKED via Server-Side Validation
Privilege Escalation: BLOCKED via Session-Based Role Management
HTTP Request Tampering: BLOCKED via Input Validation

8. Conclusion

This laboratory exercise provided a comprehensive, hands-on exploration of web application security vulnerabilities and their remediation. Working through all 16 tasks, the lab successfully demonstrated the full penetration testing lifecycle: environment setup, vulnerability identification, exploitation, documentation, and secure remediation.

8.1 Key Findings

- Four critical/high severity vulnerabilities were identified in the Flask web application: SQL Injection, Authentication Bypass, Privilege Escalation, and HTTP Request Manipulation.
- All four vulnerabilities were successfully exploited, demonstrating real-world attack scenarios in a controlled environment.
- The vulnerable application could be completely compromised by an attacker with basic knowledge of web security.

8.2 Lessons Learned

- Never trust user input: All data supplied by users must be validated, sanitized, and treated as potentially malicious before processing.
- Parameterized queries are non-negotiable: String concatenation in SQL queries creates critical injection vulnerabilities that are trivial to exploit.
- Access control must be server-side: Role assignment and privilege decisions must never rely on client-supplied parameters such as URL query strings.
- Session management is foundational: Proper session creation, maintenance, and destruction are essential components of any authenticated web application.
- Defense in depth: No single security control is sufficient. Layered security measures provide more robust protection.

8.3 Real-World Impact

The vulnerabilities demonstrated in this lab are not theoretical. SQL Injection attacks have been responsible for some of the most significant data breaches in history, including the LinkedIn breach (2012), the Yahoo breach (2013), and numerous banking and government system compromises. Authentication bypass and privilege escalation vulnerabilities similarly feature prominently in real-world incident reports.

Understanding how these attacks work—and how to prevent them—is an essential skill for any software developer, security engineer, or systems administrator. The secure coding practices implemented in Tasks 14-16 of this lab represent the minimum baseline security standards that every production web application should meet.

9. References

The following references were consulted during the design, execution, and documentation of this laboratory exercise:

Standards & Guidelines

46. OWASP Foundation. (2021). OWASP Top Ten 2021. Open Web Application Security Project. <https://owasp.org/Top10/>
47. OWASP Foundation. (2021). A03:2021 – Injection. https://owasp.org/Top10/A03_2021-Injection/
48. OWASP Foundation. (2021). A07:2021 – Identification and Authentication Failures. [https://owasp.org/Top10/A07_2021-Identification and Authentication Failures/](https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/)
49. OWASP Foundation. SQL Injection Prevention Cheat Sheet. [https://cheatsheetseries.owasp.org/cheatsheets/SQL Injection Prevention Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html)
50. OWASP Foundation. Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Framework Documentation

51. Pallets Projects. (2024). Flask Documentation (3.x). <https://flask.palletsprojects.com/>
52. Pallets Projects. (2024). Flask Security Considerations. <https://flask.palletsprojects.com/en/stable/security/>
53. Python Software Foundation. (2024). Python 3 Documentation. <https://docs.python.org/3/>
54. Microsoft Corporation. (2024). SQL Server Security Documentation. <https://docs.microsoft.com/en-us/sql/relational-databases/security/>
55. PortSwigger Web Security. (2024). Burp Suite Documentation. <https://portswigger.net/burp/documentation>

Security Tools

56. PortSwigger. (2024). Burp Suite Community Edition. <https://portswigger.net/burp/communitydownload>
57. Offensive Security. (2024). Kali Linux Documentation. <https://www.kali.org/docs/>
58. Microsoft. (2024). SQL Server Management Studio (SSMS). <https://docs.microsoft.com/en-us/sql/ssms/>

— End of Report —

Zahid Ali |Air University Islamabad, Multan Campus